```
# Exact pairing certificate over Q(sqrt5), for cycle 0, from the integer table only.
# Partner rule: different exact level, and sector components positive scalar
multiples of each other (exact).
import numpy as np, io, contextlib, sympy as sp
from sympy.polys.matrices import DomainMatrix
ns={}
with contextlib.redirect_stdout(io.StringIO()):
exec(open('45_table_only_cycle_frame.py').read(),ns)
G,B,res,N=ns['G'],ns['B'],ns['res'],ns['N']; p1=res[0][1]
K=sp.QQ.algebraic_field(sp.sqrt(5))
def esign(e):                      # exact sign of a+b*sqrt5
    a,b=[sp.Rational(t) for t in (sp.Poly(K.to_sympy(e),sp.sqrt(5)).all_coeffs()
[::-1]+[0,0])[:2]]
    if b==0: return int(sp.sign(a))
    if a==0: return int(sp.sign(b))
    if sp.sign(a)==sp.sign(b): return int(sp.sign(a))
    return int(sp.sign(a)) if a*a>5*b*b else int(sp.sign(b))
GBq=sp.Matrix(G[np.ix_(B,B)].tolist()); Cq=(GBq.inv()*sp.Matrix(G[B,:].tolist())).T
M=sp.Matrix([[Cq[p1[B[b]],i] for b in range(8)] for i in range(8)])
x=sp.symbols('x'); cp=M.charpoly(x).as_expr()
D=lambda A: DomainMatrix.from_Matrix(A).convert_to(K)
MD,GBD,CD=D(M),D(GBq),D(Cq)
def sector(f):
    F=DomainMatrix.zeros((8,8),K)
    for c in sp.Poly(f,x).all_coeffs(): F=F*MD+DomainMatrix.eye(8,K)*K.from_sympy(c)
    Kn=F.nullspace().transpose(); P=Kn*
(Kn.transpose()*GBD*Kn).inv()*Kn.transpose()*GBD   # exact GB-orthogonal projector
    return P
fs=[f for f,_ in sp.factor_list(cp,extension=sp.sqrt(5))[1]]
P=[sector(f) for f in fs]
VD=[(CD*P[s].transpose()) for s in range(2)]
VK=[[[VD[t][r,j].element for j in range(8)] for r in range(N)] for t in range(2)]   #
canonical field elements
Vs=[VD[t].to_Matrix() for t in range(2)]          # row r = exact sector-s components
of element r
GBs=GBq
def sq(v): return sp.radsimp(sp.expand((v*GBs*v.T)[0]))
lev=[[sq(Vs[s][r,:]) for r in range(N)] for s in range(2)]
s=0; Lv=list(set(lev[s])); L=Lv if esign(K.from_sympy(sp.radsimp(Lv[1]-Lv[0])))>0
else Lv[::-1]; inner=[r for r in range(N) if lev[s][r]==L[0]]; outer=[r for r in
range(N) if lev[s][r]==L[1]]
print("sector A exact levels:",L,"counts",len(inner),len(outer))
def canon(r):                                    # exact direction key in canonical
Q(sqrt5) form
    v=VK[s][r]; k=next(i for i,e in enumerate(v) if e!=K.zero); a=v[k]
    return tuple(K.to_sympy(e/a) for e in v), esign(a), k
```

```
dirs={}
for o in outer:
    key,sg,k=canon(o); dirs.setdefault((key,sg),[]).append(o)
part={}; counts=[]
for i in inner:
    key,sg,k=canon(i); hits=dirs.get((key,sg),[]); counts.append(len(hits))
    if len(hits)==1: part[i]=hits[0]
print("exact same-direction partners per inner element:",sorted(set(counts)),"|
pairs:",len(part),"| outer elements used once:",len(set(part.values()))==len(outer))
num=ns['res'];
# compare with the numerical pairing of script 46
ns2={}
with contextlib.redirect_stdout(io.StringIO()):
exec(open('46_table_only_pairing.py').read().split("# (e)")[0].split("# (c)")[0],ns2)
print("exact pairing identical to the numerical pairing:",part==ns2['part'])
print("relation value between partners (exact integers):",sorted({int(G[i,o]) for i,o
in part.items()}))
# exact scale factor in sector A and behaviour in sector B
phi=(1+sp.sqrt(5))/2
lamA={sp.radsimp(Vs[0][o,k]/Vs[0][i,k]) for i,o in part.items() for k in [next(j for
j in range(8) if Vs[0][i,j]!=0)]}
print("exact scale factor, sector A (outer = lambda * inner):",lamA,"| equals phi:",
[sp.simplify(l-phi)==0 for l in lamA])
ip=K.from_sympy(sp.radsimp(1/phi)); okB=len(part)==120 and all(all(VK[1][o]
[j]+ip*VK[1][i][j]==K.zero for j in range(8)) for i,o in part.items())
okA=len(part)==120 and all(all(VK[0][o][j]-K.from_sympy(phi)*VK[0][i][j]==K.zero for
j in range(8)) for i,o in part.items())
print('sector A exactly: partner components = phi * element components, all 120
pairs:',okA)
print("sector B exactly: partner components = -(1/phi) * element components, all 120
pairs:",okB)

# operator L = phi*P_A - (1/phi)*P_B on the full 8D space (exact)
I8=DomainMatrix.eye(8,K); f=K.from_sympy(phi); fi=K.from_sympy(sp.radsimp(1/phi))
Lop=P[0]*f - P[1]*fi
print("P_A+P_B = I:",P[0]+P[1]==I8,"| L^2 = L + I:",Lop*Lop==Lop+I8,"| L^-1 = L -
I:",Lop*(Lop-I8)==I8)
CK=[[CD[r,j].element for j in range(8)] for r in range(N)]
keyset={tuple(K.to_sympy(e) for e in CK[r]):r for r in range(N)}
img=lambda op,r: tuple(K.to_sympy(sum((op[i,j].element*CK[r][j] for j in
range(8)),K.zero)) for i in range(8))   # op acts on coefficient column
print("L maps every inner element to its partner:",all(keyset.get(img(Lop,i))==o for
i,o in part.items()))
print("L applied to outer elements lands back in the 240:",sum(keyset.get(img(Lop,o))
is not None for o in part.values()),"of 120")
Li=Lop-I8
print("L^-1 maps every outer element back to its
partner:",all(keyset.get(img(Li,o))==i for i,o in part.items()))
```